

postgreSQL

Índice

1 Configuración de PostgreSQL y pgAdmin con Docker 5

1.1 Despliegue mediante contenedores independientes 5

1.2 Configuración unificada con Docker Compose 6

2 Bases de datos y SQL 8

3 Base de datos de prueba: Pagila 10

4 SELECT y WHERE 13

4.1 La sintaxis básica de SELECT 13

4.2 Palabras clave de SELECT 14

4.3 Filtrado de registros con WHERE 15

4.4 Ejemplo de código práctico adaptado 15

5 Funciones de agregación y GROUP BY 17

5.1 Funciones de agregación en PostgreSQL 17

5.2 La cláusula GROUP BY 18

6 ORDER BY y LIMIT 21

6.1 La cláusula ORDER BY 21

6.2 La cláusula LIMIT 21

6.3 Casos de uso comunes y OFFSET 22

7 JOIN y SUBQUERIES 23

7.1 JOIN 23

7.2 SUBQUERIES (Consultas Anidadas) ... 24

8 Manipulación de fechas ... 27

9 INSERT, UPDATE y DELETE ... 30

9.1 INSERT ... 30
9.2 UPDATE ... 31
9.3 DELETE ... 32

10 Transacciones ... 34

10.1 Control de transacciones ... 34
10.2 Ejemplo de código ... 35
10.3 Casos de uso ... 35

11 Diseño relacional, Entidad-Relación y Normalización ... 36

11.1 Etapas de diseño de una base de datos relacional ... 36
11.2 El modelo Entidad-Relación ... 37
11.3 Normalización de bases de datos ... 38

12 Creación de bases de datos y tablas ... 40

12.1 Creación de una base de datos ... 40
12.2 Creación de tablas y tipos de datos ... 41
12.3 Ejemplo básico y el tipo IDENTITY ... 41
12.4 Casos de uso y valores por defecto ... 42
12.5 Creación de relaciones (Claves Foráneas) ... 42

13 Modificar tablas (ALTER TABLE) ... 44

13.1 Añadir una nueva columna ... 44
13.2 Eliminar una columna existente ... 44
13.3 Cambiar el tipo de datos de una columna ... 45
13.4 Añadir y eliminar índices ... 45

13.5 Establecer una regla de validación (CHECK) 46

14 Índices y Vistas 47

14.1 Índices 47
14.2 Vistas 48

15 Programación en Base de Datos: Funciones, Procedimientos y Triggers 51

15.1 Funciones en PostgreSQL 51
15.2 Procedimientos Almacenados (Stored Procedures) 52
15.3 Triggers (Disparadores) 53

16 Ejercicios 55

16.1 Consultas simples y filtros 55
16.2 Agrupaciones y cruces básicos 55
16.3 Múltiples JOIN, subconsultas y fechas 55
16.4 Programación en Base de Datos 56

17 Solucionario: Ejercicios de PostgreSQL (Base de datos Pagila) 57

17.1 Consultas simples y filtros 57
17.2 Agrupaciones y cruces básicos 58
17.3 Múltiples JOIN, subconsultas y fechas 58
17.4 Programación en Base de Datos 59

1 Configuración de PostgreSQL y pgAdmin con Docker

Este documento explica cómo utilizar contenedores de Docker para desplegar y configurar un servidor de PostgreSQL junto con la herramienta de administración pgAdmin 4. Se presentan dos enfoques de infraestructura: el despliegue mediante contenedores independientes y la configuración unificada mediante Docker Compose.

1.1 Despliegue mediante contenedores independientes

Podemos ejecutar los siguientes comandos en la terminal para descargar y ejecutar las imágenes oficiales en Docker. Este enfoque es útil para mantener una única instancia del gestor de base de datos que conecte con múltiples servidores.

1. Ejecución del contenedor de PostgreSQL:

TERMINAL

```
docker run -d -p 5432:5432 --name servidor-postgres -e POSTGRES_USER=postgres -e POSTGRES_PASS
WORD=mi-contraseña -e POSTGRES_DB=bootcamp_db postgres:16
```

El parámetro `-d` indica que el contenedor debe ejecutarse en segundo plano y el parámetro `-p` indica que el puerto 5432 del contenedor (que es el puerto que utiliza PostgreSQL por defecto) debe mapearse al puerto 5432 de la máquina host. Los parámetros `-e` establecen las variables de entorno obligatorias para inicializar el usuario, la contraseña y la base de datos por defecto.

2. Ejecución del contenedor de pgAdmin 4:

TERMINAL

```
docker run -d -p 5050:80 --name gestor-pgadmin -e PGADMIN_DEFAULT_EMAIL=admin@bootcamp.com -e P
GADMIN_DEFAULT_PASSWORD=admin dpage/pgadmin4
```

> *Nota de configuración: Al registrar el servidor en la interfaz de pgAdmin, no se debe utilizar `localhost` como Host name si ambos servicios se ejecutan en contenedores separados. Se deberá introducir la dirección IP de la máquina host o utilizar el DNS interno de Docker (`host.docker.internal` en sistemas compatibles).*

Puedes usar el siguiente comando para ver la IP de un contenedor:

TERMINAL

```
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' nombreContenedor
```



1.2 Configuración unificada con Docker Compose

Para asegurar que todo el entorno se despliega de forma conjunta, se utiliza un archivo de configuración personalizado usando `docker-compose`.

1.2.1 Definición en docker-compose.yml

Dentro del archivo `docker-compose.yml`, se define un servicio llamado `db` para PostgreSQL y un servicio `pgadmin` para el cliente visual:

```
services:
  db:
    container_name: servidor-postgres
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: postgres
```

```
      POSTGRES_PASSWORD: mi-contraseña
      POSTGRES_DB: bootcamp_db
    ports:
      - "5432:5432"
    volumes:
      - ./database/scripts:/docker-entrypoint-initdb.d:ro
      - postgres_data:/var/lib/postgresql/data

  pgadmin:
    container_name: gestor-pgadmin
    image: dpage/pgadmin4
    restart: unless-stopped
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@bootcamp.com
      PGADMIN_DEFAULT_PASSWORD: admin
    ports:
      - "5050:80"
    depends_on:
      - db

volumes:
  postgres_data:
```

1.2.2 Explicación de los volúmenes personalizados

- `./database/scripts:/docker-entrypoint-initdb.d:ro`: Esta línea monta un directorio local dentro del contenedor. Cualquier script `.sql` ubicado en esta ruta local se ejecutará automáticamente durante la inicialización de la base de datos. El sufijo `ro` indica que se monta en modo solo lectura.
- `postgres_data:/var/lib/postgresql/data`: Genera un volumen persistente gestionado por Docker para almacenar la información de la base de datos, garantizando que los datos no se pierdan al detener o destruir el contenedor.

Referencias

- Setting up PostgreSQL and pgAdmin 4 with Docker. Medium

2 Bases de datos y SQL

Una base de datos es un conjunto organizado de datos que se almacenan de manera persistente en un disco duro u otro medio de almacenamiento digital. Se utiliza para almacenar, organizar y acceder a grandes cantidades de datos de manera eficiente y segura. Las bases de datos se utilizan en una amplia gama de aplicaciones, desde tiendas en línea y sistemas bancarios hasta administración de registros médicos y científicos.

Las bases de datos se estructuran en tablas, que contienen filas y columnas. Las filas representan entradas individuales en la base de datos, mientras que las columnas representan los diferentes tipos de datos que se almacenan en la tabla. Cada fila en la tabla se identifica de forma única mediante una clave primaria, que es un valor que no se repite y que identifica de manera exclusiva esa fila.

En la mayoría de los sistemas de gestión de bases de datos (DBMS), como PostgreSQL, se utilizan lenguajes de consulta estructurados (SQL) para crear, consultar y modificar tablas de bases de datos. Algunos ejemplos de comandos SQL incluyen:

- **SELECT**: el comando `SELECT` se utiliza para recuperar datos de una tabla. Por ejemplo, la siguiente consulta devolverá todos los datos en la tabla "clientes" con una dirección en Nueva York:

SQL

```sql
SELECT * FROM clientes WHERE direccion = 'Nueva York';
```

- **INSERT**: el comando `INSERT` se utiliza para agregar nuevas filas a una tabla. Por ejemplo, la siguiente consulta agregará un nuevo cliente a la tabla "clientes":

SQL

```sql
INSERT INTO clientes (nombre, direccion, telefono) VALUES ('Juan', 'Chicago', '555-1234');
```

- **UPDATE**: el comando `UPDATE` se utiliza para actualizar datos en una tabla. Por ejemplo, la siguiente consulta actualizará el teléfono del cliente con ID 1234 a '555-5678':

SQL

```sql
UPDATE clientes SET telefono = '555-5678' WHERE id = 1234;
```

- **DELETE**: el comando `DELETE` se utiliza para eliminar filas de una tabla. Por ejemplo, la siguiente consulta eliminará todos los clientes con una dirección en Nueva York:

SQL

```sql
DELETE FROM clientes WHERE direccion = 'Nueva York';
```

En resumen, las bases de datos se utilizan para almacenar grandes cantidades de información de manera organizada y eficiente. SQL es un lenguaje utilizado para consultar, modificar y manipular datos en bases de datos. Los ejemplos anteriores son solo algunos de los comandos que se pueden utilizar para trabajar con bases de datos en SQL.

Referencias

- Wikipedia: Base de datos
- W3Schools: SQL Tutorial
- SQLZoo: Interactive SQL Tutorial

3 Base de datos de prueba: Pagila

La base de datos Pagila es un *port* oficial a PostgreSQL de la conocida base de datos de ejemplo Sakila. Está diseñada para mostrar la funcionalidad del motor de base de datos, así como también para proporcionar una herramienta de aprendizaje para aquellos que están aprendiendo SQL. La base de datos contiene información sobre una empresa ficticia de alquiler de películas y está dividida en varias tablas relacionales.

Las tablas principales que componen la base de datos Pagila (con sus nombres convencionales en inglés) son:

1. **actor**: Contiene información sobre los actores que actúan en las películas (actor_id, first_name, last_name, last_update).
2. **address**: Contiene información sobre las direcciones de los clientes y las tiendas donde se alquilan las películas (address_id, address, address2, district, city_id, postal_code, phone, last_update).
3. **category**: Contiene información sobre las categorías de las películas (category_id, name, last_update).
4. **city**: Contiene información sobre las ciudades donde se alquilan las películas (city_id, city, country_id, last_update).
5. **country**: Contiene información sobre los países donde se alquilan las películas (country_id, country, last_update).
6. **customer**: Contiene información sobre los clientes que alquilan las películas (customer_id, store_id, first_name, last_name, email, address_id, active, create_date, last_update).
7. **film**: Contiene información sobre las películas disponibles para alquilar (film_id, title, description, release_year, language_id, rental_duration, rental_rate, length, replacement_cost, rating, special_features, last_update).
8. **film_actor**: Contiene información sobre los actores que actúan en las películas (actor_id, film_id, last_update).
9. **film_category**: Contiene información sobre las categorías de las películas (film_id, category_id, last_update).
10.

inventory: Contiene información sobre los inventarios disponibles en cada tienda (inventory_id, film_id, store_id, last_update).

11. **language**: Contiene información sobre los idiomas utilizados en las películas (language_id, name, last_update).
12. **payment**: Contiene información sobre los pagos realizados por los clientes (payment_id, customer_id, staff_id, rental_id, amount, payment_date, last_update).
13. **rental**: Contiene información sobre los alquileres de películas realizados por los clientes (rental_id, rental_date, inventory_id, customer_id, return_date, staff_id, last_update).
14. **staff**: Contiene información sobre el personal que trabaja en las tiendas (staff_id, first_name, last_name, address_id, email, store_id, active, username, password, last_update).
15. **store**: Contiene información sobre las tiendas donde se alquilan las películas (store_id, manager_staff_id, address_id, last_update).

Las tablas están relacionadas entre sí por medio de claves foráneas (*Foreign Keys*). A continuación se describen las relaciones estructurales entre las tablas:

- **actor y film_actor**: Relación uno a muchos. Cada actor puede actuar en varias películas.
- **film y film_actor**: Relación uno a muchos. Cada película puede tener varios actores que actúen en ella.
- **category y film_category**: Relación uno a muchos. Cada categoría puede tener varias películas.
- **film y film_category**: Relación uno a muchos. Cada película puede tener varias categorías.
- **language y film**: Relación uno a muchos. Cada idioma puede estar presente en varias películas.
- **inventory y film**: Relación uno a muchos. Cada inventario puede contener varias películas.
- **store y inventory**: Relación uno a muchos. Cada tienda puede tener varios inventarios.
- **store y staff**: Relación uno a muchos. Cada tienda puede tener varios miembros del personal.
- **address y customer**: Relación uno a muchos. Cada dirección puede tener varios clientes.
- **address y staff / store**: Relación uno a muchos. Cada dirección puede tener varias tiendas.
- **city y address**: Relación uno a muchos. Cada ciudad puede tener varias direcciones.
- **country y city**: Relación uno a muchos. Cada país puede tener varias ciudades.
- **payment y rental**: Relación uno a muchos. Cada pago puede estar vinculado a varios alquileres.
- **customer y payment**: Relación uno a muchos. Cada cliente puede tener varios pagos.

- **staff y payment**: Relación uno a muchos. Cada miembro del personal puede tener varios pagos.
- **staff y rental**: Relación uno a muchos. Cada miembro del personal puede tener varios alquileres.
- **inventory, rental y customer**: Relación muchos a muchos. Cada alquiler y cada cliente pueden estar relacionados con varios inventarios.

En resumen, la base de datos Pagila contiene información sobre actores, películas, clientes, inventarios, pagos, personal y tiendas, y sus relaciones se han diseñado para reflejar la complejidad del negocio de alquiler de películas.

Referencias

- Repositorio Oficial GitHub: PostgreSQL Pagila Sample Database
- PostgreSQL Tutorial: Pagila Database

4 SELECT y WHERE

Para acceder a los datos almacenados en una base de datos, se utiliza el lenguaje de consulta estructurado (SQL). SQL es un lenguaje de programación que se utiliza para crear, editar y manipular bases de datos relacionales. Uno de los comandos más importantes en SQL es el comando `SELECT`, que se utiliza para recuperar datos de una tabla o vista en la base de datos.

4.1 La sintaxis básica de SELECT

En su forma más simple, una consulta de extracción consta de tres partes principales: la cláusula `SELECT`, la cláusula `FROM` y la cláusula `WHERE`. El formato básico de un comando es el siguiente:

SQL

```sql
SELECT columna1, columna2, columnaN
FROM tabla1
WHERE condición;
```

Donde:

- `SELECT` indica las columnas de la tabla de la que se quieren recuperar datos.
- `FROM` indica la tabla o tablas de las que se van a obtener los datos.
- `WHERE` indica la condición que deben cumplir los datos para ser recuperados.

A continuación, se muestra un ejemplo de cómo utilizar el comando para recuperar datos de una supuesta tabla "clientes":

SQL

```sql
SELECT nombre, apellido, direccion
FROM clientes
WHERE ciudad = 'Madrid';
```

Este comando devuelve los nombres, apellidos y direcciones de todos los clientes que viven en Madrid.

4.2 Palabras clave de SELECT

En PostgreSQL se pueden utilizar diferentes palabras clave en la cláusula `SELECT` para obtener diferentes resultados, tales como:

- **DISTINCT**: Devuelve valores únicos de la columna seleccionada, eliminando duplicados en los resultados.

SQL

```sql
SELECT DISTINCT nombre_columna FROM nombre_tabla;
```

- **COUNT**: Devuelve el número de registros en una tabla o el número de registros que cumplen cierta condición.

SQL

```sql
SELECT COUNT(*) FROM nombre_tabla;
```

- **SUM**: Devuelve la suma total de los valores numéricos en una columna determinada.

SQL

```sql
SELECT SUM(nombre_columna) FROM nombre_tabla;
```

- **MAX y MIN**: Devuelve el valor máximo y mínimo de la columna seleccionada.

SQL

```sql
SELECT MAX(nombre_columna) FROM nombre_tabla;
```

4.3 Filtrado de registros con WHERE

`WHERE` es una cláusula fundamental en SQL que se utiliza para filtrar registros en una consulta `SELECT`. Con `WHERE` se puede especificar una condición estricta para seleccionar únicamente los registros que la cumplan.

Se puede combinar `WHERE` con múltiples operadores lógicos y de comparación. Los operadores que se pueden utilizar incluyen:

- `=` (igual a)
- `<>` o `!=` (diferente a)
- `>` (mayor que) y `<` (menor que)
- `>=` (mayor o igual que) y `<=` (menor o igual que)
- `BETWEEN` (entre un rango de valores, inclusivo)
- `IN` (para buscar si un valor coincide con los elementos de una lista)
- `LIKE` (para hacer coincidencias parciales de texto mediante patrones).

> ***El operador `ILIKE`:*** *En PostgreSQL, el operador LIKE es estricto y sensible a mayúsculas. Para realizar búsquedas ignorando mayúsculas, PostgreSQL dispone del operador ILIKE.*

4.4 Ejemplo de código práctico adaptado

Supongamos que tenemos una tabla llamada "customers" en una base de datos de compras en línea, y queremos seleccionar los nombres y las ubicaciones de los clientes que han realizado una compra en los últimos 30 días.

La consulta en **PostgreSQL** se escribiría de la siguiente forma:

SQL

```sql
SELECT name, location
FROM customers
WHERE date_purchased >= NOW() - INTERVAL '30 days';
```

Este comando selecciona los campos requeridos donde la fecha de la compra es igual o superior a los últimos 30 días. Nótese la sintaxis específica de PostgreSQL para intervalos de tiempo (`INTERVAL '30 days'`), la cual requiere que el valor del intervalo vaya entre comillas simples.

Referencias

- PostgreSQL Official Documentation: Queries - SELECT
- PostgreSQL Official Documentation: The WHERE Clause
- W3Schools: SQL WHERE Clause

5 Funciones de agregación y GROUP BY

5.1 Funciones de agregación en PostgreSQL

En PostgreSQL podemos utilizar diversas funciones de agregación para obtener resúmenes de los datos almacenados en nuestras tablas. Las funciones más comunes son `COUNT()`, `SUM()`, `AVG()`, `MIN()` y `MAX()`.

- **COUNT()**: Se utiliza para contar el número de filas que existe en una tabla. Es muy útil para saber la cantidad de registros que se tienen en una base de datos.

SQL

```sql
SELECT COUNT(*) FROM ventas;
```

Este ejemplo devuelve el número total de registros que se tienen en la tabla de ventas.

- **SUM()**: Se utiliza para sumar los valores numéricos de una columna especificada en una tabla.

SQL

```sql
SELECT SUM(cantidad) FROM ventas;
```

Este ejemplo devuelve la suma total de la cantidad vendida en la tabla de ventas.

- **AVG()**: Se utiliza para obtener el promedio (media aritmética) de los valores de una columna especificada.

SQL

```sql
SELECT AVG(precio) FROM productos;
```

Este ejemplo devuelve el promedio de precio de todos los productos almacenados en la tabla correspondiente.

- **MIN()**: Se utiliza para obtener el valor mínimo de una columna especificada.

SQL

```sql
SELECT MIN(precio) FROM productos;
```

Devuelve el precio mínimo de un producto almacenado en la tabla de productos.

- **MAX()**: Se utiliza para obtener el valor máximo de una columna especificada.

SQL

```sql
SELECT MAX(precio) FROM productos;
```

Devuelve el precio máximo de un producto almacenado en la tabla de productos.

Estas funciones son muy útiles en el ámbito empresarial para generar reportes de ventas, control de inventarios y análisis de estadísticas.

5.2 La cláusula GROUP BY

`GROUP BY` es una cláusula utilizada para agrupar filas en una tabla en función de los valores de una o varias columnas. Se utiliza principalmente de forma conjunta con las funciones de agregado mencionadas anteriormente (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) para calcular y resumir datos basados en dichos grupos.

La sintaxis básica de `GROUP BY` es la siguiente:

SQL

```sql
SELECT columna1, columna2, funcion_agregada(columna3)
FROM tabla
GROUP BY columna1, columna2;
```

Aquí, `columna1` y `columna2` son las columnas que se utilizarán para agrupar las filas, y `funcion_agregada(columna3)` es una función que se utilizará para calcular valores consolidados para cada grupo.

5.2.1 Ejemplo práctico de agrupación

Supongamos que tenemos una tabla llamada "ordenes" que contiene información sobre las operaciones de compra realizadas por los clientes. Podemos utilizar `GROUP BY` para obtener el total de ventas acumulado por cada cliente:

SQL

```sql
SELECT cliente, SUM(total)
FROM ordenes
GROUP BY cliente;
```

En este ejemplo, utilizamos la función de agregado `SUM()` para calcular el total monetario de ventas por cliente. Los resultados se agruparon por la columna "cliente".

5.2.2 Agrupación estadística y tratamiento de fechas en PostgreSQL

Otro caso de uso común de `GROUP BY` es el cálculo de datos estadísticos agrupados por periodos de tiempo .

Para calcular la cantidad de órdenes realizadas cada mes en PostgreSQL, la consulta se redactaría de la siguiente manera:

SQL

```sql
SELECT EXTRACT(MONTH FROM fecha) AS mes, COUNT(id)
FROM ordenes
GROUP BY EXTRACT(MONTH FROM fecha);
```

En este ejemplo, utilizamos la función de fecha `EXTRACT(MONTH FROM fecha)` para aislar el mes de la fecha de cada orden. Luego, utilizamos la función de agregado `COUNT()` para contar la cantidad de operaciones por mes .

En resumen, `GROUP BY` es una cláusula fundamental que se utiliza para agrupar filas y realizar cálculos estadísticos en los datos, permitiendo resumir la información en función de un criterio de agrupación específico .

Referencias

- PostgreSQL Official Documentation: Aggregate Functions
- PostgreSQL Official Documentation: The GROUP BY Clause

6 ORDER BY y LIMIT

`ORDER BY` y `LIMIT` son cláusulas muy útiles en PostgreSQL para ordenar y limitar la cantidad de resultados arrojados por una consulta. El uso combinado de ambas es fundamental para optimizar el rendimiento de la base de datos y mejorar la presentación de los datos en las aplicaciones cliente.

6.1 La cláusula ORDER BY

La cláusula `ORDER BY` se utiliza para ordenar el resultado de una consulta por una o más columnas según el criterio especificado. Por defecto, el ordenamiento se realiza de forma ascendente (`ASC`), aunque se puede especificar el orden descendente utilizando la palabra clave `DESC` .

Por ejemplo, si queremos ordenar una lista de estudiantes por su nombre de forma alfabética, utilizaríamos la siguiente consulta:

SQL

```sql
SELECT * FROM estudiantes ORDER BY nombre;
```

El resultado sería una lista de estudiantes ordenada alfabéticamente por su nombre. Si quisiéramos ordenar la lista desde la Z hasta la A, simplemente añadiríamos `DESC` al final de la consulta (`ORDER BY nombre DESC`).

6.2 La cláusula LIMIT

Por otro lado, la cláusula `LIMIT` se utiliza para limitar la cantidad de resultados devueltos por una consulta. Esto es especialmente importante en tablas con millones de registros, donde devolver todos los datos saturaría la memoria del servidor y la red.

Por ejemplo, si queremos obtener solamente los 5 primeros estudiantes en la lista ordenada alfabéticamente por su nombre, utilizaríamos la siguiente consulta:

SQL

```sql
SELECT * FROM estudiantes ORDER BY nombre LIMIT 5;
```

El resultado sería una lista con los primeros 5 estudiantes ordenados alfabéticamente por su nombre.

6.3 Casos de uso comunes y OFFSET

En resumen, la cláusula `ORDER BY` permite ordenar el resultado de una consulta según una o varias columnas, mientras que `LIMIT` permite limitar la cantidad de resultados obtenidos.

Casos de uso comunes para estas cláusulas son, por ejemplo, para la paginación de resultados (utilizando `LIMIT` y `OFFSET`), o para obtener los resultados más recientes o más antiguos (utilizando `ORDER BY` y las funciones de fecha y hora de PostgreSQL).

Nota sobre la paginación: En el desarrollo de aplicaciones web, es habitual combinar `LIMIT` con la cláusula `OFFSET`. Mientras que `LIMIT` define cuántos registros mostrar, `OFFSET` define cuántos registros saltarse antes de empezar a mostrar resultados. Por ejemplo, `LIMIT 10 OFFSET 20` mostraría los resultados del 21 al 30, lo cual equivaldría a la página 3 de una vista paginada.

7 JOIN y SUBQUERIES

7.1 JOIN

En PostgreSQL, el comando `JOIN` se utiliza para combinar filas de dos o más tablas basadas en una condición común. Puedes usar `JOIN` para unir dos o más tablas y recuperar datos de ambas tablas al mismo tiempo, lo cual es fundamental en el diseño de bases de datos relacionales.

Existen cuatro tipos principales de `JOIN` soportados por el estándar SQL en PostgreSQL:

- **INNER JOIN**: Muestra solo los registros que tienen una coincidencia en ambas tablas que están siendo unidas.
- **LEFT JOIN** (o LEFT OUTER JOIN): Muestra todos los registros de la tabla izquierda y los registros coincidentes de la tabla derecha. Si no hay coincidencia, el resultado mostrará valores `NULL` del lado derecho.
- **RIGHT JOIN** (o RIGHT OUTER JOIN): Muestra todos los registros de la tabla derecha y los registros coincidentes de la tabla izquierda.
- **FULL OUTER JOIN**: Incluye todos los registros de ambas tablas, coincidentes o no coincidentes.

A continuación, se muestran las estructuras sintácticas para cada tipo de `JOIN`:

INNER JOIN:

SQL

```sql
SELECT *
FROM tabla1
INNER JOIN tabla2
ON tabla1.columna = tabla2.columna;
```

LEFT JOIN:

SQL

```sql
SELECT *
FROM tabla1
LEFT JOIN tabla2
ON tabla1.columna = tabla2.columna;
```

RIGHT JOIN:

SQL

```sql
SELECT *
FROM tabla1
RIGHT JOIN tabla2
ON tabla1.columna = tabla2.columna;
```

FULL OUTER JOIN:

SQL

```sql
SELECT *
FROM tabla1
FULL OUTER JOIN tabla2
ON tabla1.columna = tabla2.columna;
```

Un caso de uso común para `JOIN` es cuando tienes dos tablas que contienen información relacionada pero no quieres unir la información en una sola tabla debido a la redundancia de datos. En cambio, puedes unir las tablas utilizando una condición común para obtener información correlacionada de ambas tablas. Por ejemplo, podrías tener una tabla de pedidos y una tabla de productos, y unir las dos tablas basándose en el identificador del producto para obtener información de pedidos y productos juntos en una sola consulta.

Es importante recordar que `JOIN` puede ser costoso en términos de rendimiento, especialmente cuando se unen tablas con grandes cantidades de datos. Es necesario optimizar las consultas y utilizar índices para mejorar el rendimiento.



7.2 SUBQUERIES (Consultas Anidadas)

Las *subqueries* (también conocidas como consultas anidadas) en SQL son consultas dentro de otras consultas principales. Se utilizan para extraer datos de una tabla base, y luego usar los resultados de esa consulta para filtrar o limitar los resultados de la consulta principal.

Existen dos tipos de *subqueries*:

- **Correlativas**: Que dependen de una tabla externa a la subconsulta.
- **No correlativas**: Que pueden ejecutarse de manera independiente a la consulta principal.

Ejemplo de subconsulta correlativa:

SQL

```sql
SELECT *
FROM orders
WHERE customer_id IN (SELECT customer_id FROM customers WHERE country = 'Mexico');
```

En este caso, la subconsulta depende de la tabla `customers` y se utiliza para limitar los resultados de la consulta principal en la tabla `orders`.

Ejemplo de subconsulta no correlativa:

SQL

```sql
SELECT *
FROM orders
WHERE order_date > (SELECT MAX(order_date) FROM orders) - 7;
```

En este caso, la subconsulta no tiene ninguna dependencia en una tabla externa y se utiliza para calcular una fecha límite que se usa para limitar los resultados de la consulta principal en la tabla `orders`.

Las *subqueries* son útiles en situaciones como:

- Consultas complejas donde se requiere filtrar en múltiples niveles o aplicar varias restricciones.
- Consultas que involucran cálculos complejos y subconjuntos de datos.
- Consultas que requieren resultados que no se pueden obtener con una simple consulta `SELECT`.

En resumen, las *subqueries* son un recurso importante y útil en SQL para procesar datos complejos y proporcionar respuestas incluyendo múltiples niveles de información.

Referencias

- W3Resource: SQL Subqueries
- PostgreSQL Official Documentation: Queries - Table Expressions (JOIN)

8 Manipulación de fechas

En PostgreSQL, la manipulación de fechas se lleva a cabo mediante una serie de funciones y operadores integrados en el motor. Antes de empezar a trabajar con fechas, es fundamental asegurarse de que el tipo de dato utilizado en la definición de la tabla sea el adecuado. PostgreSQL utiliza un estándar muy riguroso para el manejo del tiempo, siendo los tipos principales `date` (solo fecha), `timestamp` (fecha y hora sin zona horaria) y `timestamptz` (fecha y hora con zona horaria, siendo este último el más recomendado para aplicaciones web modernas).

Las funciones y operadores nativos más comunes para trabajar con estos tipos de datos son:

- **`CURRENT_DATE`**: Devuelve la fecha actual del sistema en formato estándar (YYYY-MM-DD).
- **`NOW()`** o **`CURRENT_TIMESTAMP`**: Devuelve la fecha y hora actuales del sistema.
- **Operadores `+` y `-` con `INTERVAL`**: Permiten agregar o restar una cantidad específica de tiempo (días, meses, años, horas) a una fecha.
- **Función `AGE()`**: Calcula la diferencia exacta entre dos fechas y devuelve un intervalo completo detallado en años, meses y días.
- **Restas de fechas**: Al restar directamente dos campos de tipo `date` utilizando el operador `-`, PostgreSQL devuelve un número entero que representa la cantidad de días de diferencia.
- **`TO_CHAR()`**: Formatea la fecha y hora convirtiéndola en texto según el patrón de salida deseado (por ejemplo, 'DD/MM/YYYY').
- **`EXTRACT()`**: Extrae un componente específico de una fecha o timestamp (como el año, el mes o el día) y lo devuelve como un valor numérico.

A continuación, se presentan varios ejemplos prácticos que ilustran cómo realizar operaciones comunes de fechas utilizando la sintaxis estándar de PostgreSQL:

Obtener la edad de una persona a partir de su fecha de nacimiento PostgreSQL simplifica enormemente este cálculo gracias a su función `AGE()`. Si queremos extraer solo el número del año para obtener la edad exacta en un formato numérico simple, la combinamos con `EXTRACT`:

SQL

```sql
SELECT EXTRACT(YEAR FROM AGE(fecha_nacimiento)) AS edad
FROM personas;
```

Listar los empleados que cumplen años este mes Para comparar únicamente el mes actual con el mes de nacimiento, extraemos ese componente de ambas fechas:

SQL

```sql
SELECT * FROM empleados
WHERE EXTRACT(MONTH FROM fecha_nacimiento) = EXTRACT(MONTH FROM CURRENT_DATE);
```

Calcular la fecha de vencimiento de una factura Si necesitamos calcular una fecha de vencimiento sumando un plazo de pago (por ejemplo, 30 días) a la fecha de emisión, utilizamos el operador de suma junto con un `INTERVAL`:

SQL

```sql
SELECT fecha_emision + INTERVAL '30 days' AS fecha_vencimiento
FROM facturas
WHERE id_factura = 1;
```

Obtener el número de días que faltan para una fecha específica Para saber cuántos días faltan para un evento, como el fin de año, basta con restar la fecha actual a la fecha objetivo. Es importante utilizar `::date` para indicar al motor que la cadena de texto proporcionada debe tratarse estrictamente como una fecha:

SQL

```sql
SELECT '2026-12-31'::date - CURRENT_DATE AS dias_para_fin_de_ano;
```

El dominio de la manipulación de fechas en PostgreSQL pasa por asimilar el uso del estándar `INTERVAL` y de herramientas robustas como `EXTRACT` y `AGE()`. Familiarizarse con ellas permite realizar cálculos temporales complejos de manera muy eficiente directamente desde la base de datos.

Referencias

- PostgreSQL Official Documentation: Date/Time Functions and Operators

9 INSERT, UPDATE y DELETE

9.1 INSERT

El comando `INSERT` en PostgreSQL se utiliza para insertar datos en una tabla existente en una base de datos. Para insertar datos, debemos especificar la tabla y los valores que deseamos agregar. A continuación, se muestra la sintaxis básica del comando `INSERT`:

SQL

```sql
INSERT INTO nombre_de_la_tabla (columna1, columna2, columna3, ...)
VALUES (valor1, valor2, valor3, ...);
```

Donde "nombre_de_la_tabla" es el nombre de la tabla en la que deseamos insertar los datos, y "columna1", "columna2", "columna3", son los nombres de las columnas a las que deseamos agregar valores. Los valores que deseamos agregar se proporcionan en el orden correspondiente de las columnas.

A continuación se muestran algunos ejemplos de cómo utilizar el comando `INSERT`:

1. Insertar un solo registro en una tabla

SQL

```sql
INSERT INTO estudiantes (nombre, edad, email)
VALUES ('Juan Pérez', 20, 'juan.perez@ejemplo.com');
```

2. Insertar varios registros en una tabla

SQL

```sql
INSERT INTO estudiantes (nombre, edad, email)
VALUES ('Pedro López', 22, 'pedro.lopez@ejemplo.com'),
       ('Ana García', 21, 'ana.garcia@ejemplo.com');
```

3. Insertar un registro en una tabla con campos nulos

SQL

```sql
INSERT INTO estudiantes (id, nombre, edad, email)
VALUES (1, 'Luisa Torres', NULL, 'luisa.torres@ejemplo.com');
```

En este caso, estamos insertando un registro en la tabla "estudiantes" con un valor `NULL` en la columna "edad".

> ***Particularidad de PostgreSQL: La cláusula RETURNING***
> *Una de las grandes ventajas de PostgreSQL es la capacidad de devolver los datos del registro que se acaba de insertar, actualizar o eliminar en la misma consulta. Si añadimos `RETURNING *` (o el nombre de una columna, como `RETURNING id`) al final de nuestro `INSERT`, el motor nos devolverá el ID autogenerado inmediatamente, ahorrando una consulta extra desde el backend.*

9.2 UPDATE

La cláusula `UPDATE` se utiliza en PostgreSQL para modificar datos existentes en una tabla. La sintaxis básica es la siguiente:

SQL

```sql
UPDATE nombre_tabla
SET columna1 = valor1, columna2 = valor2, ...
WHERE condicion;
```

- `nombre_tabla`: el nombre de la tabla que se va a actualizar.
- `columna1, columna2, ...`: las columnas que se van a actualizar.

- `valor1, valor2, ...`: los nuevos valores que se van a asignar a las columnas.
- `condicion`: una cláusula `WHERE` opcional que especifica qué filas deben actualizarse.

Ejemplo de actualización de una columna: Si queremos cambiar el email de una usuaria llamada Maria, podemos hacerlo de la siguiente manera:

SQL

```sql
UPDATE usuarios
SET email = 'maria@ejemplo.com'
WHERE nombre = 'Maria';
```

Ejemplo de actualización múltiple: Si queremos cambiar el nombre y el apellido de un usuario con ID 3 al mismo tiempo:

SQL

```sql
UPDATE usuarios
SET nombre = 'Juan', apellido = 'Perez'
WHERE id = 3;
```

Es crítico recordar que la cláusula `WHERE` es opcional, pero si no se utiliza, se actualizarán todas las filas de la tabla de forma indiscriminada.

9.3 DELETE

La instrucción `DELETE` en PostgreSQL se utiliza para eliminar uno o varios registros de una tabla en una base de datos. La sintaxis básica es la siguiente:

SQL

```sql
DELETE FROM nombre_tabla WHERE condicion;
```

Donde `nombre_tabla` es el nombre de la tabla de la que se desean eliminar los registros y `condicion` es la condición que deben cumplir los registros para ser eliminados.

Ejemplo de código: Si quisiéramos eliminar el registro de la usuaria Ana podríamos hacerlo de la siguiente manera:

SQL

```sql
DELETE FROM usuarios WHERE nombre = 'Ana';
```

Al igual que en la actualización, si no se especifica una condición con `WHERE`, se eliminarán todos los registros de la tabla. La instrucción `DELETE` elimina información de manera permanente, así que es necesario tener precaución al utilizarla y siempre realizar copias de seguridad previas.

Referencias

- PostgreSQL Official Documentation: INSERT
- PostgreSQL Official Documentation: UPDATE
- PostgreSQL Official Documentation: DELETE
- PostgreSQL Official Documentation: RETURNING Clause

10 Transacciones

En PostgreSQL, una transacción es una secuencia de operaciones que se ejecutan como una sola unidad lógica e indivisible, en la que las operaciones tienen efecto como si se estuvieran ejecutando en el mismo instante de tiempo. En otras palabras, una transacción es una serie de operaciones que deben tratarse como si fueran una sola, de modo que el éxito de la misma implique que todas las operaciones se han completado correctamente, mientras que cualquier fallo en cualquiera de las operaciones debe suponer un rechazo absoluto de todas las operaciones previas dentro de ese bloque.

Las transacciones permiten asegurar que la base de datos siempre esté en un estado consistente, y proporcionan mecanismos seguros para recuperarse de posibles errores.

10.1 Control de transacciones

En PostgreSQL, el control manual de las transacciones se realiza mediante el uso de las sentencias `BEGIN` (el estándar principal en Postgres, equivalente a `START TRANSACTION`), `COMMIT` y `ROLLBACK`.

- **BEGIN**: Indica al motor de la base de datos que se va a comenzar una nueva transacción. Todas las sentencias DML que se ejecuten a partir de este punto no serán visibles para otros usuarios hasta que se confirmen.
- **COMMIT**: Indica que se debe confirmar la transacción, es decir, que todas las operaciones se han completado con éxito y los cambios deben guardarse permanentemente en el disco.
- **ROLLBACK**: Indica que se debe deshacer la transacción. Es decir, si alguna de las operaciones ha fallado, se revierte el estado de la base de datos exactamente a como estaba antes de ejecutar la sentencia `BEGIN`.

10.2 Ejemplo de código

SQL

```sql
BEGIN;

INSERT INTO tabla1 (campo1, campo2) VALUES ('valor1', 'valor2');
UPDATE tabla2 SET campo3 = 'valor3' WHERE campo4 = 'valor4';

COMMIT;
```

En este ejemplo estamos iniciando una transacción mediante la instrucción `BEGIN`, dentro de la cual se están realizando dos operaciones: la inserción de un registro en `tabla1` y la actualización de un registro en `tabla2`. Si ambas operaciones se completan con éxito, se ejecuta la sentencia `COMMIT`, que confirma la transacción y la hace definitiva.

Si ocurriera un error durante el `UPDATE` (por ejemplo, una violación de clave foránea o una caída del sistema), el motor detendría la ejecución. En ese caso, el desarrollador o la aplicación ejecutaría la sentencia `ROLLBACK`, que deshace la transacción completa, eliminando también el `INSERT` previo que sí había funcionado, manteniendo así la integridad.

10.3 Casos de uso

Los casos de uso más comunes para las transacciones en bases de datos relacionales son aquellos en los que se necesite hacer varias operaciones juntas, asegurando que si alguna falla, no se apliquen las demás, manteniendo siempre la integridad de los datos.

Por ejemplo, es obligatorio utilizar transacciones en operaciones bancarias (restar saldo de una cuenta y sumarlo a otra), reservas de billetes, compras en línea y otras acciones críticas en las que se necesite garantizar la coherencia absoluta de los datos.

Referencias

- PostgreSQL Official Documentation: Transactions

11 Diseño relacional, Entidad-Relación y Normalización

El diseño de una base de datos relacional es un paso crítico previo a la escritura de código SQL. Un buen diseño garantiza el rendimiento, evita la redundancia de datos y asegura la escalabilidad del sistema a largo plazo.

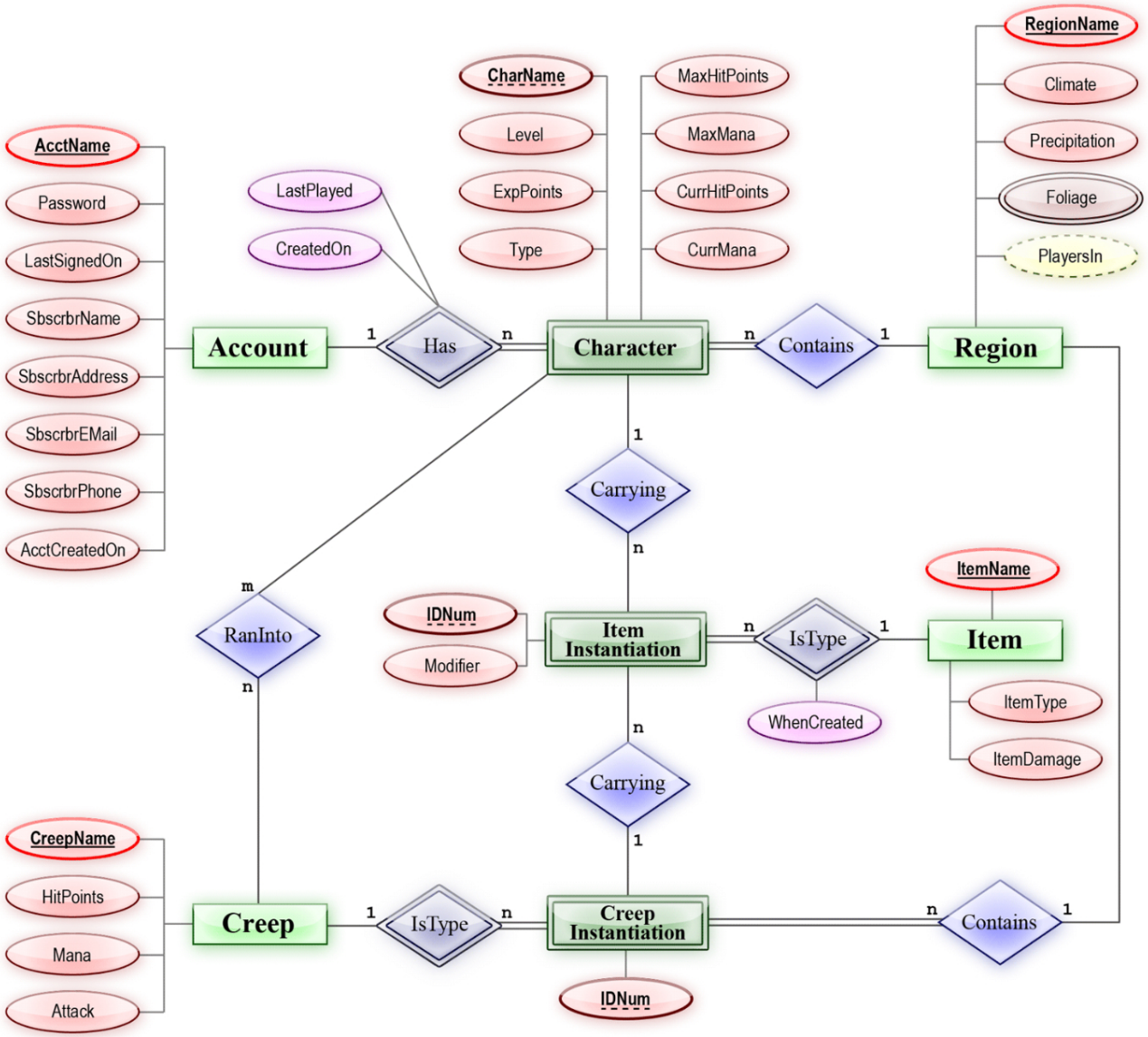
11.1 Etapas de diseño de una base de datos relacional

El diseño de una base de datos relacional para PostgreSQL consta de las siguientes etapas:

1. **Análisis de requisitos**: En esta etapa se deben identificar las necesidades y requerimientos del usuario, es decir, qué información se desea almacenar y cómo se relaciona entre sí. Además, también se deben definir las entidades y atributos que formarán parte de la base de datos.
2. **Diseño conceptual**: Con base en el análisis de requisitos, se realiza un modelo ER (Entidad-Relación) que representa la estructura de la base de datos sin tener en cuenta los detalles de implementación específicos del motor. El modelo ER consta de entidades, relaciones entre ellas y atributos, y sirve como guía para la creación del modelo lógico.
3. **Diseño lógico**: En esta etapa se traduce el modelo ER a un modelo lógico. Se definen las tablas que formarán la base de datos y se definen las columnas de cada tabla. También se definen las relaciones (claves primarias y foráneas) entre las tablas.
4. **Diseño físico**: En esta etapa se define la forma en que se almacenarán los datos en el disco. Se definen los tipos de datos y tamaños de cada columna y se establecen restricciones de integridad para garantizar la consistencia de los datos.
5. **Implementación**: Con base en el diseño físico, se procede a la creación de la base de datos en PostgreSQL. La implementación también incluye la creación de las tablas, la definición de las relaciones, la creación de índices y la definición de los privilegios de acceso para los usuarios.
6.

Mantenimiento: Finalmente, el mantenimiento implica la realización de copias de seguridad, la optimización, la modificación de tablas y la adición de nuevas características conforme las necesidades evolucionan.

11.2 El modelo Entidad-Relación



El modelo entidad-relación es una herramienta de modelamiento de datos que permite representar de manera gráfica las entidades (objetos o conceptos) y las relaciones que existen entre ellos. Existen tres tipos principales de entidades en este modelo:

1. **Entidades concretas**: son objetos físicos que pueden ser tocados o vistos en el mundo real, como por ejemplo una persona, un auto, una casa o un libro.
2.

Entidades abstractas: son objetos que no podemos tocar o ver, pero que existen en el mundo real, como por ejemplo un pedido, una venta o una factura.

3. **Entidades asociativas**: son objetos que se utilizan para representar relaciones de muchos a muchos entre otras entidades.

Por otro lado, las relaciones que representa este modelo pueden ser de varios tipos:

- **Relaciones uno a uno (1:1)**: ocurre cuando una entidad se relaciona directamente con otra entidad y no hay más entidades involucradas.
- **Relaciones uno a muchos (1:N)**: ocurre cuando una entidad se relaciona con varias entidades de otro tipo, pero estas últimas solo tienen una relación con la entidad en cuestión.
- **Relaciones muchos a uno (N:1)**: ocurre cuando varias entidades se relacionan con otra entidad de otro tipo.
- **Relaciones muchos a muchos (N:M)**: cuando varias entidades se relacionan con varias entidades de otro tipo.

Por ejemplo, en una base de datos de una tienda de libros, tenemos dos entidades, "libro" y "autor", donde cada libro tiene un único autor y cada autor puede publicar varios libros (relación 1:N) .

11.3 Normalización de bases de datos

La normalización de bases de datos relacionales es una técnica que consiste en organizar la información de una base de datos en estructuras lógicas que eliminan la redundancia y aseguran la integridad de los datos. Existen distintos niveles de normalización, siendo los más importantes el primero, segundo y tercero:

- **1NF (Primera Forma Normal)**: Establece que cada tabla debe tener una clave primaria y que todas las columnas deben contener valores atómicos. Los valores de una columna no pueden ser arrays, objetos o estructuras complejas.
- **2NF (Segunda Forma Normal)**: Una tabla cumple la 2NF si cumple con la 1NF y, además, cada columna no clave depende completamente de la clave primaria. Esto evita la redundancia parcial.
- **3NF (Tercera Forma Normal)**: Una tabla cumple la 3NF si cumple con la 2NF y, además, no existe ninguna dependencia transitiva entre las columnas no clave.

11.3.1 Ejemplo de código (Adaptado a PostgreSQL)

A continuación, se muestra un ejemplo de cómo definir tablas normalizadas en PostgreSQL:

SQL

```sql
-- Definición de las tablas (3NF)
CREATE TABLE alumnos (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    edad INT NOT NULL,
    carrera VARCHAR(50) NOT NULL
);

CREATE TABLE materias (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    creditos INT NOT NULL
);

-- Entidad asociativa para resolver la relación N:M
CREATE TABLE inscripciones (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_alumno INT NOT NULL,
    id_materia INT NOT NULL,
    semestre INT NOT NULL,
    FOREIGN KEY (id_alumno) REFERENCES alumnos(id),
    FOREIGN KEY (id_materia) REFERENCES materias(id)
);
```

En este ejemplo, se definen tres tablas: `alumnos`, `materias` e `inscripciones`. La tabla `inscripciones` actúa como una tabla de relación (entidad asociativa) entre ambas, indicando qué estudiante se ha matriculado en qué asignatura, cumpliendo con la 3NF al no existir dependencias entre las columnas no clave.

Referencias

- Wikipedia: Modelo entidad-relación
- Lucidchart: Qué es un diagrama entidad-relación
- Wikipedia: Normalización de bases de datos

12 Creación de bases de datos y tablas

Para crear bases de datos SQL se necesita utilizar un Sistema Gestor de Bases de Datos (SGBD) que permita crear, gestionar y manipular las bases de datos. Algunos de los SGBD más utilizados en la industria son PostgreSQL, MySQL, Oracle y Microsoft SQL Server.

En este capítulo nos centraremos en el Lenguaje de Definición de Datos (DDL), que nos permite construir la estructura física donde se almacenará nuestra información.

12.1 Creación de una base de datos

Para crear una base de datos en PostgreSQL desde cero, se siguen los siguientes pasos lógicos:

1. **Conexión**: Abrir el SGBD (como pgAdmin 4 o la terminal `psql`) y conectarse al servidor o motor de bases de datos.
2. **Creación**: Crear una nueva base de datos y asignarle un nombre utilizando la sentencia `CREATE DATABASE`. Ejemplo:

SQL

```sql
CREATE DATABASE bootcamp_db;
```

3. **Selección de la base de datos**:

Para cambiar de base de datos en la terminal de Postgres (`psql`) se utiliza el metacomando `\c mi_base_de_datos`. Si se utiliza pgAdmin 4, simplemente se debe hacer clic izquierdo sobre el nombre de la nueva base de datos en el árbol de navegación lateral para abrir una herramienta de consultas (*Query Tool*) conectada directamente a ella.

12.2 Creación de tablas y tipos de datos

Para crear tablas en PostgreSQL se utiliza el comando `CREATE TABLE`. Este comando requiere de la definición detallada de las columnas y sus respectivos tipos de datos.

La sintaxis básica para crear una tabla es la siguiente:

SQL

```sql
CREATE TABLE nombre_tabla (
    nombre_columna1 tipo_dato_requerido restricciones,
    nombre_columna2 tipo_dato_requerido restricciones
);
```

12.3 Ejemplo básico y el tipo IDENTITY

A la hora de crear la tabla debemos especificar su nombre, los nombres y tipos de datos de las columnas y las claves primarias. A continuación, se presenta un ejemplo de cómo crear una tabla simple para usuarios utilizando la sintaxis recomendada:

SQL

```sql
CREATE TABLE usuarios (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL,
    telefono VARCHAR(20)
);
```

En este ejemplo:

- **`id`**: Es de tipo entero (`INT`) y utiliza la cláusula `GENERATED ALWAYS AS IDENTITY`. Esta es la forma moderna y estándar en SQL para crear un número que se autoincrementa de forma segura, evitando que se puedan insertar IDs manualmente por error.
- **`nombre`**: Es de tipo `VARCHAR` con una longitud máxima de 50 caracteres y marcado como `NOT NULL` (obligatorio).
- **`telefono`**: Es de tipo `VARCHAR` con una longitud máxima de 20 caracteres y admite valores nulos al no tener restricción.

12.4 Casos de uso y valores por defecto

Es importante tener en cuenta que elegir el tipo de dato adecuado para cada columna es fundamental para el correcto funcionamiento de la aplicación y la eficiencia del almacenamiento.

Ejemplo de tabla para productos: Una aplicación de compras en línea podría crear una tabla llamada productos que contenga información sobre los productos que se venden.

SQL

```sql
CREATE TABLE productos (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    descripcion TEXT,
    precio DECIMAL(10,2) NOT NULL,
    disponible BOOLEAN DEFAULT true
);
```

En PostgreSQL, el tipo `BOOLEAN` es nativo y real (acepta `true`, `false` o `null`). La cláusula `DEFAULT true` permite definir valores predeterminados para una columna si no se especifica en el `INSERT`.

12.5 Creación de relaciones (Claves Foráneas)

Para crear relaciones entre tablas debemos especificar una clave foránea en una tabla que haga referencia a la clave primaria de otra tabla. Para ello podemos utilizar la sentencia `FOREIGN KEY`.

Supongamos un escenario con una tabla `estudiantes` y una tabla `asignaturas`. Para relacionarlas mediante una tabla intermedia `matriculas`, la sintaxis sería la siguiente:

SQL

```sql
CREATE TABLE matriculas (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    codigo_estudiante INT,
    codigo_asignatura INT,
    FOREIGN KEY (codigo_estudiante) REFERENCES estudiantes (id),
    FOREIGN KEY (codigo_asignatura) REFERENCES asignaturas (id)
);
```

En las claves foráneas hemos especificado que `codigo_estudiante` hace referencia a la columna `id` de la tabla `estudiantes` y `codigo_asignatura` hace referencia a la columna `id` de la tabla

`asignaturas`. Las claves foráneas son útiles en el momento de relacionar tablas y para mantener la integridad referencial de los datos almacenados.

Referencias

- PostgreSQL Official Documentation: CREATE DATABASE
- PostgreSQL Official Documentation: CREATE TABLE
- PostgreSQL Official Documentation: Data Types

13 Modificar tablas (ALTER TABLE)

La alteración de tablas en PostgreSQL es un proceso mediante el cual se modifican las características de una tabla ya existente en una base de datos. Entre las operaciones más comunes que se pueden realizar mediante la alteración de tablas se encuentran:

- Añadir o eliminar columnas.
- Cambiar el tipo de datos de una columna.
- Añadir o eliminar restricciones (Constraints) y reglas de validación.

A continuación, se describen algunos casos de uso comunes para la alteración de tablas en PostgreSQL, junto con ejemplos de código para llevar a cabo estas operaciones.

13.1 Añadir una nueva columna

En el siguiente ejemplo se muestra cómo añadir una nueva columna a una tabla existente en PostgreSQL:

SQL

```sql
ALTER TABLE tabla1 ADD COLUMN columna1 INT;
```

En este ejemplo se añade una nueva columna llamada "columna1" a la tabla "tabla1" con un tipo de datos de número entero (`INT`).

13.2 Eliminar una columna existente

En este ejemplo se muestra cómo eliminar una columna de una tabla existente:

SQL

```sql
ALTER TABLE tabla1 DROP COLUMN columna1;
```

En este ejemplo se elimina de forma permanente la columna "columna1" de la tabla "tabla1". Todos los datos contenidos en esa columna se perderán.

13.3 Cambiar el tipo de datos de una columna

En el estándar SQL implementado por PostgreSQL, la sintaxis correcta exige el uso de `ALTER COLUMN` seguido de la palabra clave `TYPE`.

En el siguiente ejemplo se muestra cómo cambiar el tipo de datos de una columna existente:

SQL

```sql
ALTER TABLE tabla1 ALTER COLUMN columna1 TYPE VARCHAR(50);
```

En este ejemplo se cambia el tipo de datos de la columna "columna1" en la tabla "tabla1" de número entero (`INT`) a cadena de caracteres (`VARCHAR`) con una longitud máxima de 50 caracteres. *(Nota: PostgreSQL intentará hacer la conversión automáticamente. Si los datos existentes no son compatibles, requerirá una cláusula `USING` adicional para indicar cómo transformar los datos).*

13.4 Añadir y eliminar índices

En PostgreSQL, aunque se pueden gestionar restricciones (como `PRIMARY KEY` o `UNIQUE`) mediante `ALTER TABLE`, la gestión de índices de rendimiento estándar se realiza de manera independiente mediante los comandos `CREATE INDEX` y `DROP INDEX`.

Para añadir un índice:

SQL

```sql
CREATE INDEX nombre_indice ON tabla1 (columna1);
```

Para eliminar un índice:

SQL

```sql
DROP INDEX nombre_indice;
```

Profundizaremos en la creación y el uso estratégico de los índices en el siguiente capítulo de este temario.

13.5 Establecer una regla de validación (CHECK)

En el siguiente ejemplo se muestra cómo establecer una regla de validación en una columna de una tabla existente:

SQL

```sql
ALTER TABLE tabla1 ADD CONSTRAINT nombre_restriccion CHECK (columna1 > 0);
```

En este ejemplo se establece una restricción llamada "nombre_restriccion" en la tabla "tabla1" que se asegura de que los valores insertados o actualizados en la columna "columna1" sean estrictamente mayores que cero.

En resumen, la alteración de tablas es una herramienta muy útil para modificar tablas existentes y adaptarlas a las necesidades de la aplicación. Es importante señalar que la alteración de una tabla puede ser una operación costosa en términos de recursos si se realiza en una tabla con una gran cantidad de registros. Por esta razón, se recomienda hacer copias de seguridad regulares antes de realizar cualquier cambio estructural importante.

Referencias

- PostgreSQL Official Documentation: ALTER TABLE
- PostgreSQL Official Documentation: Constraints

14 Índices y Vistas

En esta fase del diseño y manejo de bases de datos relacionales, nos centraremos en herramientas avanzadas que permiten optimizar el rendimiento de las consultas y establecer capas de abstracción y seguridad sobre nuestros datos.

14.1 Índices

Un índice en PostgreSQL es una estructura de datos que se utiliza para aumentar la velocidad de la búsqueda y el rendimiento en las tablas de la base de datos. Básicamente, se crea un índice en una o más columnas de una tabla para permitir búsquedas más rápidas. Los índices se pueden crear en columnas individuales o en múltiples columnas de una tabla.

Cuando se crea un índice, se construye una estructura adicional (por defecto en PostgreSQL, un árbol B o *B-Tree*) que contiene los valores de la columna indexada y los punteros a las filas correspondientes en la tabla.

Hay algunos casos de uso en los que es estrictamente recomendable agregar un índice a una tabla para mejorar el rendimiento en las consultas:

- Tablas con grandes conjuntos de datos.
- Tablas que se utilizan con frecuencia para consultas y búsquedas (especialmente en columnas usadas frecuentemente en las cláusulas `WHERE` o `JOIN`).
- Tablas que tienen datos que se actualizan con poca frecuencia.

Para crear un índice en una tabla, se utiliza la siguiente sintaxis:

SQL

```sql
CREATE INDEX nombre_indice ON nombre_tabla (columna_indice);
```

Donde:

- `nombre_indice`: Es el nombre que se le desea dar al índice.
- `nombre_tabla`: Es el nombre de la tabla en la que se desea crear el índice.
- `columna_indice`: Nombre de la columna o columnas de la tabla en la que queremos crear el índice.

Por ejemplo, si queremos crear un índice en la columna `nombre` de la tabla `usuarios`, podemos utilizar el siguiente código:

SQL

```sql
CREATE INDEX idx_nombre ON usuarios (nombre);
```

También es posible crear índices en múltiples columnas de una tabla. Por ejemplo, si tenemos una tabla `productos` con las columnas `id`, `nombre`, `precio` y `fecha_disponible`, podemos crear un índice en las columnas `precio` y `fecha_disponible` de la siguiente manera:

SQL

```sql
CREATE INDEX idx_precio_fecha ON productos (precio, fecha_disponible);
```

Esta sintaxis en particular se utiliza para crear índices compuestos, es decir, índices que utilizan varias columnas.

Finalmente, es importante tener en cuenta que, aunque los índices mejoran el rendimiento de las consultas y búsquedas en la base de datos, también pueden afectar negativamente el rendimiento de la inserción o actualización de registros en la tabla, ya que el motor tiene que actualizar la estructura del índice en cada operación de escritura. Por ello, es vital analizar cuidadosamente qué columnas requieren índices.

14.2 Vistas

En PostgreSQL, las vistas son objetos que contienen una consulta SQL en sí mismas y se utilizan para simplificar el acceso a datos complejos a través de una consulta más simple y con un nombre asignado. Básicamente, una vista es una tabla virtual que representa los resultados de una consulta SQL subyacente.

Las vistas pueden ser utilizadas para:

- **Simplificar consultas complejas**: las consultas SQL que involucran múltiples tablas (`JOIN`) y condiciones pueden ser difíciles de entender y de mantener. Las vistas pueden simplificar estas consultas al permitir que el usuario final pueda acceder a un conjunto más reducido y lógicamente estructurado de datos.
- **Establecer niveles de seguridad**: Las vistas pueden utilizarse para restringir el acceso a determinados datos sensibles. Por ejemplo, se podría crear una vista sobre una tabla que sólo muestre los datos públicos relevantes, dejando fuera los datos confidenciales (como contraseñas o información financiera). De esta forma, las personas tienen acceso a los datos necesarios sin exponer información delicada.
- **Compartir datos**: Las vistas pueden ser utilizadas para compartir conjuntos de datos complejos con diversos usuarios o aplicaciones que no necesariamente tienen conocimientos técnicos para crear y ejecutar consultas complejas en SQL.

Para crear una vista, se utiliza la sintaxis `CREATE VIEW`. Por ejemplo, si quisiéramos crear una vista para mostrar los artículos de una tienda en línea que tienen una cantidad mayor o igual a 10, se escribiría una consulta SQL que filtre los productos que cumplan la condición y se crearía una vista con esa consulta:

SQL

```sql
CREATE VIEW tienda_view AS
SELECT * FROM articulos WHERE cantidad >= 10;
```

Una vez creada la vista, podemos utilizarla en cualquier consulta SQL como si se tratara de una tabla normal:

SQL

```sql
SELECT * FROM tienda_view WHERE precio > 100;
```

Es importante notar que las vistas estándar no almacenan datos, sino que son simplemente una "vista virtual" de los datos reales que se encuentran en las tablas subyacentes. Cada vez que se hace una consulta a la vista, se ejecuta en tiempo real la consulta SQL que la definió.

En PostgreSQL, si deseamos modificar la consulta interna de una vista existente, la práctica recomendada es utilizar `CREATE OR REPLACE VIEW`. Si se desea eliminar la vista completamente del sistema, se utiliza `DROP VIEW`.

Referencias

- PostgreSQL Official Documentation: Indexes
- PostgreSQL Official Documentation: Views

15 Programación en Base de Datos: Funciones, Procedimientos y Triggers

En PostgreSQL, podemos programar lógica de negocio directamente dentro del motor de base de datos utilizando SQL combinado con su lenguaje procedural nativo, conocido como PL/pgSQL. Esto nos permite crear rutinas que procesan información a una velocidad mucho mayor que si extrajéramos los datos a nuestro entorno de backend para procesarlos allí.

15.1 Funciones en PostgreSQL

En las bases de datos relacionales, las funciones son subrutinas que pueden recibir parámetros y realizar operaciones compactas y repetitivas. Estas se crean con el objetivo de retornar un valor específico tras su ejecución.

Para crear una función en PostgreSQL utilizando PL/pgSQL, la sintaxis estructurada requiere delimitar el cuerpo de la función con `$$` y especificar el lenguaje al final de la declaración:

SQL

```sql
CREATE OR REPLACE FUNCTION nombre_funcion (parametro1 tipo, parametro2 tipo)
RETURNS tipo_dato_retorno AS $$
DECLARE
    -- Declaración de variables internas
BEGIN
    -- Cuerpo de la función y lógica
    RETURN resultado;
END;
$$ LANGUAGE plpgsql;
```

Ejemplo práctico: A continuación, se muestra cómo estructurar una función matemática sencilla que calcula el promedio de tres números numéricos:

SQL

```sql
CREATE OR REPLACE FUNCTION calcular_promedio(num1 INT, num2 INT, num3 INT)
RETURNS FLOAT AS $$
DECLARE
```

```
    promedio FLOAT;
BEGIN
    promedio := (num1 + num2 + num3) / 3.0;
    RETURN promedio;
END;
$$ LANGUAGE plpgsql;
```

Una vez creada la función, podemos llamarla directamente en una consulta utilizando la instrucción `SELECT`:

SQL

```
SELECT calcular_promedio(5, 10, 15); -- Devuelve 10.0
```



15.2 Procedimientos Almacenados (Stored Procedures)

Un procedimiento almacenado es un conjunto de instrucciones SQL que se almacenan en el servidor de bases de datos y se pueden invocar y ejecutar posteriormente utilizando un nombre específico. Entre los beneficios de los procedimientos almacenados están la reutilización del código, la encapsulación de la lógica de la base de datos y la mejora del rendimiento.

A diferencia de las funciones (que siempre deben retornar un valor), los procedimientos están diseñados para ejecutar tareas (inserciones, actualizaciones masivas) y permiten gestionar transacciones internas, como usar `COMMIT` o `ROLLBACK` dentro de su propio bloque.

Para crear un procedimiento almacenado en PostgreSQL, se utiliza la siguiente sintaxis:

SQL

```
CREATE OR REPLACE PROCEDURE insert_user(p_name VARCHAR, p_email VARCHAR)
AS $$
BEGIN
    INSERT INTO users (name, email) VALUES (p_name, p_email);
END;
$$ LANGUAGE plpgsql;
```

Para invocar el procedimiento almacenado y ejecutar su lógica interna, se debe utilizar obligatoriamente la instrucción `CALL` en lugar de un `SELECT` tradicional:

SQL

```
CALL insert_user('Juan', 'juan@example.com');
```

15.3 Triggers (Disparadores)

Los triggers son objetos de bases de datos que se utilizan para ejecutar acciones automáticas en respuesta a ciertos eventos que se producen en una tabla en particular. Los triggers se ejecutan antes (`BEFORE`) o después (`AFTER`) de que se realice una de las siguientes acciones: `INSERT` , `UPDATE` o `DELETE` .

En cuanto a los casos de uso más comunes, destacan:

- **Verificación de datos**: llevar a cabo comprobaciones de los datos que se están insertando para garantizar que cumplan con reglas o restricciones complejas.
- **Auditoría**: registrar información sobre alteraciones realizadas en una tabla, guardándolas en un historial o log de auditoría.
- **Sincronización de datos**: sincronizar y replicar datos automáticamente entre diferentes tablas relacionadas.

15.3.1 El proceso de creación en dos pasos

En PostgreSQL, un trigger está compuesto estrictamente por dos elementos que deben crearse por separado: un procedimiento (la función) que se ejecuta como respuesta al evento, y el propio evento que activa el trigger vinculándolo a la tabla.

Paso 1: Crear la función del trigger Primero debemos crear una función que retorne el tipo especial `TRIGGER` . Utilizaremos como ejemplo un registro de auditoría:

SQL

```
CREATE OR REPLACE FUNCTION auditar_cambios()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO log (usuario, fecha, tabla, accion)
    VALUES (CURRENT_USER, CURRENT_TIMESTAMP, TG_TABLE_NAME, TG_OP);

    RETURN NEW; -- En un trigger AFTER o BEFORE INSERT/UPDATE, se debe retornar NEW.
END;
$$ LANGUAGE plpgsql;
```

PostgreSQL provee variables especiales automáticas dentro de estas funciones, como `TG_TABLE_NAME` para obtener el nombre de la tabla afectada y `TG_OP` para identificar la operación realizada.

Paso 2: Vincular el trigger a la tabla Una vez existe la función, creamos el disparador que la invocará tras el evento deseado:

SQL

```sql
CREATE TRIGGER trig_auditar_cambios
AFTER INSERT ON nombre_tabla
FOR EACH ROW
EXECUTE FUNCTION auditar_cambios();
```

Con este modelo estructurado en dos pasos, PostgreSQL permite reutilizar la misma función (por ejemplo, `auditar_cambios()`) en múltiples triggers de diferentes tablas, reduciendo drásticamente la duplicación de código en el sistema.

Referencias

- Documentación oficial de PostgreSQL: PL/pgSQL - SQL Procedural Language
- Documentación oficial de PostgreSQL: CREATE PROCEDURE
- Documentación oficial de PostgreSQL: CREATE TRIGGER

16 Ejercicios

16.1 Consultas simples y filtros

- **Ejercicio:** Obtener un listado de todos los clientes activos. El resultado debe mostrar el nombre, los apellidos y el correo electrónico, ordenados alfabéticamente por apellido. *(Tablas implicadas: customer)*
- **Ejercicio:** Mostrar el título, el año de lanzamiento y la duración de las 10 películas más largas del catálogo. *(Tablas implicadas: film)*
- **Ejercicio:** Encontrar todas las películas que tengan un coste de reemplazo (*replacement_cost*) mayor a 20 dólares y que estén calificadas como 'PG-13'. *(Tablas implicadas: film)*

16.2 Agrupaciones y cruces básicos

- **Ejercicio:** Calcular el total de ingresos (suma del *amount*) generados por cada empleado (*staff_id*). *(Tablas implicadas: payment)*
- **Ejercicio:** ¿Cuántas películas hay de cada categoría? Mostrar el nombre de la categoría y el recuento total de películas, ordenado de mayor a menor cantidad. *(Tablas implicadas: category, film_category)*
- **Ejercicio:** Mostrar el nombre y apellidos de los clientes junto con la ciudad en la que viven. *(Tablas implicadas: customer, address, city)*

16.3 Múltiples JOIN, subconsultas y fechas

- **Ejercicio:** Encontrar el título de todas las películas en las que ha actuado la actriz 'PENELOPE GUINESS'. *(Tablas implicadas: actor, film_actor, film)*

- **Ejercicio:** Obtener una lista de los clientes que **nunca** han alquilado una película. Intenta resolverlo utilizando una subconsulta (Subquery). *(Tablas implicadas: customer, rental)*
- **Ejercicio:** Utilizando la función `EXTRACT()`, mostrar en qué mes (independientemente del año) se han realizado más alquileres en la historia del videoclub. *(Tablas implicadas: rental)*
- **Ejercicio:** Insertar un nuevo cliente en la base de datos y, en la misma consulta, hacer que PostgreSQL devuelva el `customer_id` que se le acaba de asignar automáticamente. *(Tablas implicadas: customer)*

16.4 Programación en Base de Datos

- **Ejercicio:** Crear una función en `PL/pgSQL` llamada `total_gastado_cliente` que reciba como parámetro el ID de un cliente (`customer_id`) y devuelva la suma total de dinero que ha gastado en el videoclub.
- **Ejercicio:** Crear un procedimiento almacenado llamado `actualizar_email_cliente` que reciba el ID de un cliente y un nuevo correo electrónico, y actualice el registro correspondiente.
- **Ejercicio:** Diseñar un Trigger estructurado en dos pasos. Primero, crear una tabla vacía llamada `auditoria_peliculas` (con columnas: titulo_pelicula, fecha_insercion, usuario). Después, crear la función y el disparador para que cada vez que se haga un `INSERT` en la tabla `film`, se guarde automáticamente un registro en la tabla de auditoría indicando qué película se ha añadido y cuándo.

17 Solucionario: Ejercicios de PostgreSQL (Base de datos Pagila)

17.1 Consultas simples y filtros

Obtener un listado de todos los clientes activos. El resultado debe mostrar el nombre, los apellidos y el correo electrónico, ordenados alfabéticamente por apellido.

SQL

```sql
SELECT first_name, last_name, email
FROM customer
WHERE active = 1
ORDER BY last_name ASC;
```

Mostrar el título, el año de lanzamiento y la duración de las 10 películas más largas del catálogo.

SQL

```sql
SELECT title, release_year, length
FROM film
ORDER BY length DESC
LIMIT 10;
```

Encontrar todas las películas que tengan un coste de reemplazo mayor a 20 dólares y que estén calificadas como 'PG-13'.

SQL

```sql
SELECT title, replacement_cost, rating
FROM film
WHERE replacement_cost > 20
  AND rating = 'PG-13';
```

17.2 Agrupaciones y cruces básicos

Calcular el total de ingresos generados por cada empleado.

SQL

```sql
SELECT staff_id, SUM(amount) AS total_ingresos
FROM payment
GROUP BY staff_id;
```

¿Cuántas películas hay de cada categoría? Mostrar el nombre de la categoría y el recuento total de películas, ordenado de mayor a menor cantidad.

SQL

```sql
SELECT c.name AS categoria, COUNT(fc.film_id) AS total_peliculas
FROM category c
JOIN film_category fc ON c.category_id = fc.category_id
GROUP BY c.name
ORDER BY total_peliculas DESC;
```

Mostrar el nombre y apellidos de los clientes junto con la ciudad en la que viven.

SQL

```sql
SELECT cu.first_name, cu.last_name, ci.city
FROM customer cu
JOIN address a ON cu.address_id = a.address_id
JOIN city ci ON a.city_id = ci.city_id;
```

17.3 Múltiples JOIN, subconsultas y fechas

Encontrar el título de todas las películas en las que ha actuado la actriz 'PENELOPE GUINESS'.

SQL

```sql
SELECT f.title
FROM film f
JOIN film_actor fa ON f.film_id = fa.film_id
```

```sql
JOIN actor a ON fa.actor_id = a.actor_id
WHERE a.first_name = 'PENELOPE'
  AND a.last_name = 'GUINESS';
```

Obtener una lista de los clientes que nunca han alquilado una película utilizando una subconsulta.

SQL

```sql
SELECT first_name, last_name
FROM customer
WHERE customer_id NOT IN (
    SELECT customer_id FROM rental
);
```

Utilizando la función EXTRACT(), mostrar en qué mes (independientemente del año) se han realizado más alquileres en la historia del videoclub.

SQL

```sql
SELECT EXTRACT(MONTH FROM rental_date) AS mes, COUNT(rental_id) AS total_alquileres
FROM rental
GROUP BY EXTRACT(MONTH FROM rental_date)
ORDER BY total_alquileres DESC
LIMIT 1;
```

Insertar un nuevo cliente en la base de datos y hacer que PostgreSQL devuelva el `customer_id` asignado automáticamente.

SQL

```sql
-- Nota: address_id y store_id deben existir en la base de datos para no dar error de clave
foránea
INSERT INTO customer (store_id, first_name, last_name, email, address_id, activebool)
VALUES (1, 'NUEVO', 'CLIENTE', 'nuevo@bootcamp.com', 1, true)
RETURNING customer_id;
```



17.4 Programación en Base de Datos

Crear una función en PL/pgSQL llamada `total_gastado_cliente` que reciba como parámetro el ID de un cliente y devuelva la suma total de dinero que ha gastado.

SQL

```sql
CREATE OR REPLACE FUNCTION total_gastado_cliente(p_customer_id INT)
RETURNS DECIMAL AS $$
DECLARE
    total DECIMAL;
BEGIN
    SELECT SUM(amount) INTO total
    FROM payment
    WHERE customer_id = p_customer_id;

    -- Manejo de nulos por si el cliente no tiene pagos
    IF total IS NULL THEN
        total := 0;
    END IF;

    RETURN total;
END;
$$ LANGUAGE plpgsql;

-- Para probarla: SELECT total_gastado_cliente(1);
```

Crear un procedimiento almacenado llamado `actualizar_email_cliente` que reciba el ID de un cliente y un nuevo correo electrónico, y actualice el registro correspondiente.

SQL

```sql
CREATE OR REPLACE PROCEDURE actualizar_email_cliente(p_customer_id INT, p_nuevo_email VARCHAR)
AS $$
BEGIN
    UPDATE customer
    SET email = p_nuevo_email
    WHERE customer_id = p_customer_id;
END;
$$ LANGUAGE plpgsql;

-- Para probarlo: CALL actualizar_email_cliente(1, 'email_actualizado@bootcamp.com');
```

Diseñar un Trigger estructurado en dos pasos para auditar las nuevas películas.

SQL

```sql
-- 1. Crear la tabla de auditoría (DDL)
CREATE TABLE auditoria_peliculas (
    titulo_pelicula VARCHAR(255),
    fecha_insercion TIMESTAMP,
    usuario VARCHAR(50)
);

-- 2. Crear la función del trigger
CREATE OR REPLACE FUNCTION auditar_nueva_pelicula()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO auditoria_peliculas (titulo_pelicula, fecha_insercion, usuario)
    VALUES (NEW.title, CURRENT_TIMESTAMP, CURRENT_USER);
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 3. Crear el disparador asociado a la tabla film
CREATE TRIGGER trig_auditoria_film
AFTER INSERT ON film
FOR EACH ROW
EXECUTE FUNCTION auditar_nueva_pelicula();

-- Para probarlo: INSERT INTO film (title, language_id) VALUES ('PELICULA DE PRUEBA', 1);
```